Lab 4 - Potpourri

Sheila Yu

2026-03-26

Introduction

In this lab you'll review and get practice with a variety of concepts, methods, and tools you've encountered thus far.

Part 1 - All about Quarto

Question 1

a. Add each of strings below to the code chunk provided in your document, render the document, and determine if the string is a proper code chunk label. If not, explain why and describe how you could fix it so the document renders.

b. Chunk label 1:

```
#| label: a-label-with-a-line-break
```

This is not a proper chunk label because it is split across two lines. Chunk labels must be defined on a single line, so the code can be fixed by combining the label into one line, such as:

```
#| label: a-label-with-a-line-break
```

- Chunk label 2:

```
#| label: areaaaaaaaaaaaaaaaaaaaaaaallllllllllllllllllyyyyyloooooooooooooooooonglabel
```

This chunk label works.

- Chunk label 3:

```
#| label: label_with_spaces
```

This chunk label will not work as Quarto treats labels like IDs, so spaces break things. To fix this, we could use underscore or dashes to connect different words, such as:

```
#| label: label_with_spaces
```

- Chunk label 4:

```
#| label: label-with-dashes
```

This chunk label works.

> **Tip**
>
> Try each label option in the code chunk provided in your document. If it gives you an error (document doesn't render), you know it's not a proper chunk label.

a. Which of the chunk label options above is the best option? Explain your reasoning in 1-2 sentences.

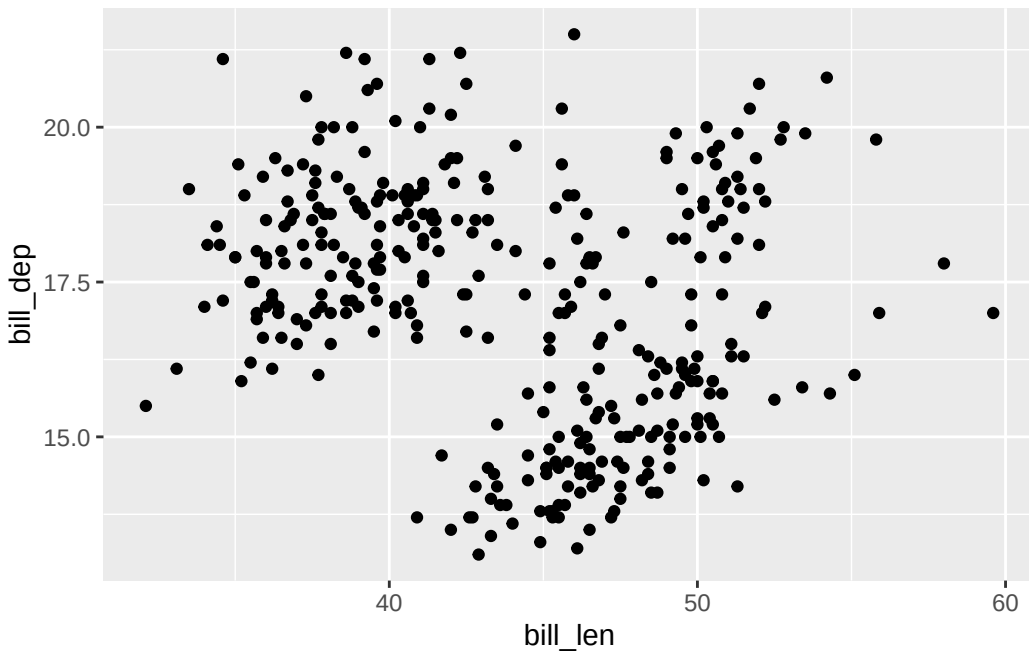
    a. Chunk label 4 is the best option as it functions properly, and it is clear, concise and easy to read.

Question 2

You have the following code chunk:

```
ggplot(penguins, aes(x = bill_len, y = bill_dep)) +
  geom_point()
```

```
Warning: Removed 2 rows containing missing values or values outside the scale range
(`geom_point()`).
```



Add the following code chunk options, one at a time, and set each to `false` and then to `true`. After each value, render your document and observe its effect. Ultimately, choose the values that are the most appropriate for this code chunk. Based on the behaviors you observe, describe what each code chunk option does.

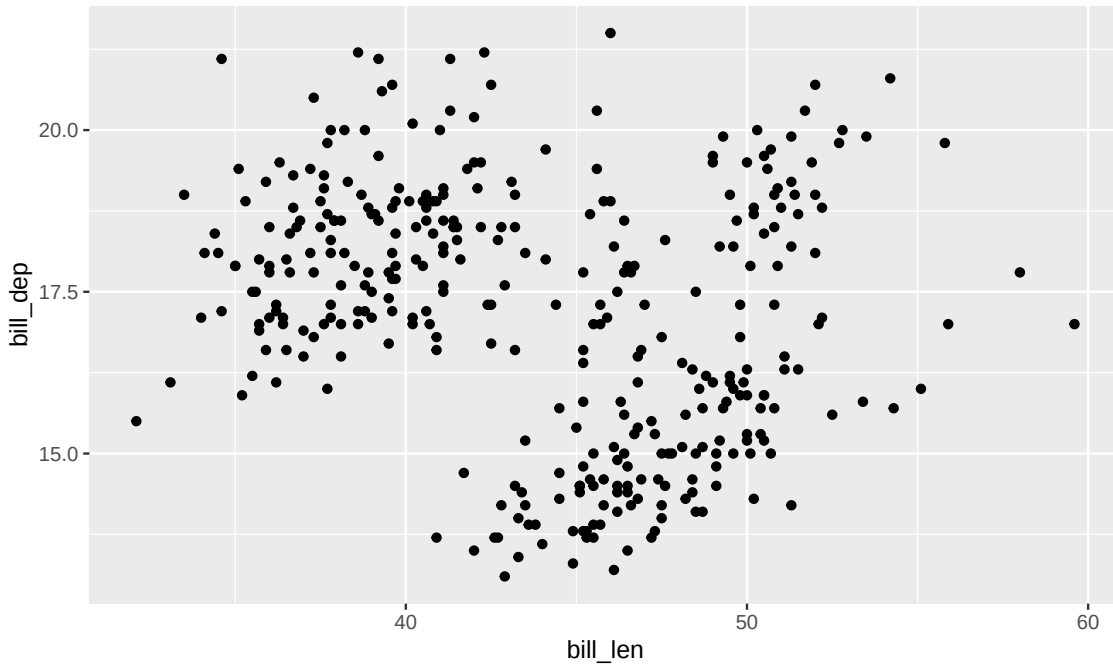
- `echo`
  - When echo = true, the code is shown in the document, whereas when echo = false, the code is hidden and only the output (the graph) is shown.
- `warning`
  - When warning = true, the warning message is shown in the document, where as when warning is false, it hides the warning messages.
- `eval`
  - when eval is true, the document runs the code, when eval is false, the document doesn't run the code

Question 3

a. You have the following code chunk again.

```
ggplot(penguins, aes(x = bill_len, y = bill_dep)) +
  geom_point()
```

```
Warning: Removed 2 rows containing missing values or values outside the scale range
(`geom_point()`).
```



Add `fig-width` and `fig-asp` as code chunk options. Try setting `fig-width` to values between 1 and 10. Try setting `fig-asp` to values between 0.1 and 1. Re-render the document after each value and observe its effect. Ultimately, choose values that make the plot look visually pleasing in the rendered document. Based on the behavior you observe, describe what each chunk option does.

The fig-width option controls how wide the figure appears in the rendered document, while fig-asp controls the aspect ratio of the figure, or how tall it appears relative to its width. Increasing fig-width makes the plot wider, and increasing fig-asp makes the plot taller.

a. You have the following code chunk, but look carefully, it's not exactly the same!

```
gplot(penguins, aes(x = bill_len, y = bill_dep) +
  geom_point()
```

```
Error in parse(text = input): <text>:3:0: unexpected end of input
1: gplot(penguins, aes(x = bill_len, y = bill_dep) +
2:   geom_point()
  ^
```

Add `error` as a code chunk option and set it to `false` and then set it to `true`. After each value, render your document and observe its effect. Ultimately, choose the value that

allows you to render your document without altering the code. Based on the behavior you observe, describe what this code chunk option does.

The error chunk option controls whether errors in a code chunk stop the document from rendering. When error = FALSE, the document stops rendering if an error occurs. When error = TRUE, the document continues rendering and displays the error message in the output.

> **Tip**
>
> Reading the documentation might also be hepful.

Render, commit, and push one last time. Make sure that you commit and push all changed documents and your Git pane is completely empty before proceeding.

Part 2 - Misrepresentation

Question 4

The following chart was shared by @GraphCrimes on X/Twitter on September 3, 2022.

a. What is misleading about this graph?

    a. The graph is misleading because the stacked bar segments are not clearly scaled or aligned to a consistent baseline, making it difficult to accurately compare the proportions across categories.

b. Suppose you wanted to recreate this plot, with improvements to avoid its misleading pitfalls from part (a). You would obviously need the data from the survey in order to be able to do that. How many observations would this data have? How many variables (at least) should it have, and what should those variables be?

    a. The dataset should contain one observation per respondent for each service, in the format of multiple rows per respondent. At minimum, it would include variables identifying the respondent, the service being evaluated, and their response category (public, private, or don't know). There is 1858 participants and 4 categories, so it should have a minimum of 7432 individual data points.

c. Load the data for this survey from `data/survation.csv`. Confirm that the data match the percentages from the visualization. That is, calculate the percentages of public sector, private sector, don't know for each of the services and check that they match the percentages from the plot.

```r
#didn't receive the data file along with the lab so I created the following based on the info

survation <- bind_rows(
  tibble(
    service = "Rail",
    response = c(
      rep("Public sector", 1747),
      rep("Private sector", 56),
      rep("Don't know", 55)
    )
  ),
  tibble(
    service = "Water",
    response = c(
      rep("Public sector", 1747),
      rep("Private sector", 56),
      rep("Don't know", 55)
    )
  ),
  tibble(
    service = "Energy",
    response = c(
      rep("Public sector", 1616),
      rep("Private sector", 111),
      rep("Don't know", 131)
    )
  ),
  tibble(
    service = "Royal Mail",
    response = c(
      rep("Public sector", 1579),
      rep("Private sector", 130),
      rep("Don't know", 149)
    )
  )
)

survation %>%
  group_by(service, response) %>%
  summarise(n = n(), .groups = "drop") %>%
  group_by(service) %>%
  mutate(percent = round(n / sum(n) * 100, 0)) %>%
```

```r
  arrange(service, desc(percent))
```

```
# A tibble: 12 x 4
# Groups:   service [4]
   service    response           n percent
   <chr>      <chr>          <int>   <dbl>
 1 Energy     Public sector   1616      87
 2 Energy     Don't know       131       7
 3 Energy     Private sector   111       6
 4 Rail       Public sector   1747      94
 5 Rail       Don't know        55       3
 6 Rail       Private sector    56       3
 7 Royal Mail Public sector   1579      85
 8 Royal Mail Don't know       149       8
 9 Royal Mail Private sector   130       7
10 Water      Public sector   1747      94
11 Water      Don't know        55       3
12 Water      Private sector    56       3
```

Please note that the value are consistent with the data from the plot as the csv datasheet is missing.

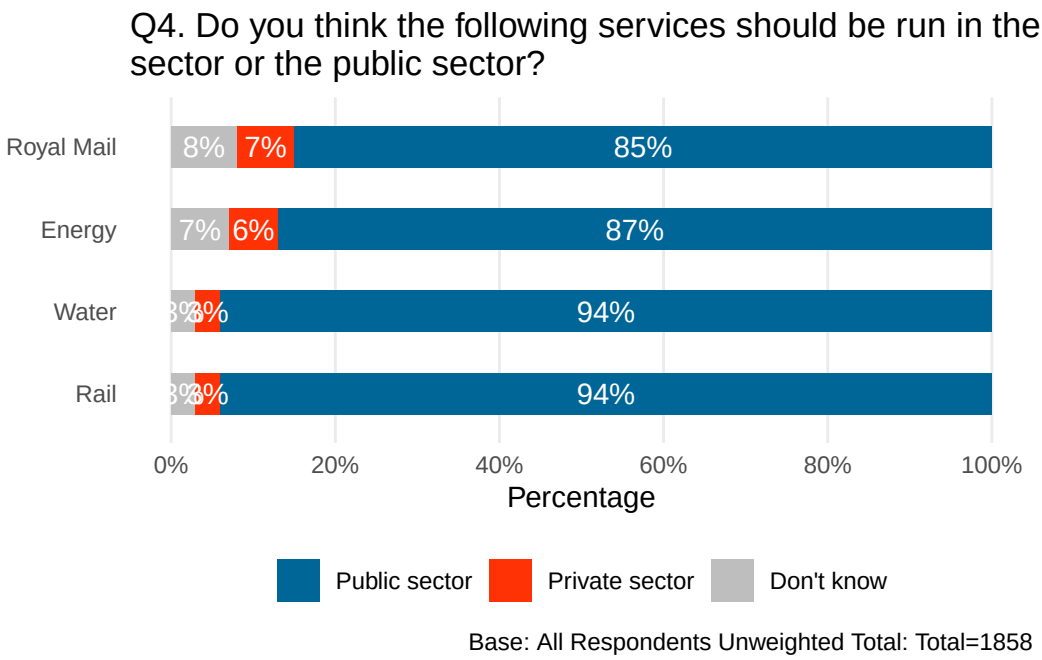
Question 5

Create an improved version of the visualization. Your improved visualization:

- should also be a stacked bar chart with services on the y-axis, presented in the same order as the original plot, and services to create the segments of the plot, and presented in the same order as the original plot
- should have the same legend location
- should have the same title and caption
- does not need to have a bolded title or a gray background

How does the improved visualization look different than the original? Does it send a different message at a first glance?

```r
survation %>%
  group_by(service, response) %>%
  summarise(n = n(), .groups = "drop") %>%
  group_by(service) %>%
  mutate(percent = n / sum(n) * 100) %>%
  mutate(
    service = factor(service, levels = c("Rail", "Water", "Energy", "Royal Mail")),
    response = factor(response,
                      levels = c("Public sector", "Private sector", "Don't know"))
  ) %>%
  ggplot(aes(x = percent, y = service, fill = response)) +
  geom_col(width = 0.5) +
  geom_text(aes(label = paste0(round(percent), "%")),
            position = position_stack(vjust = 0.5),
            color = "white",
            size = 4) +
  scale_fill_manual(
    values = c(
      "Public sector" = "#006697",
      "Private sector" = "#FF3205",
      "Don't know" = "gray"
    )
  ) +
  scale_x_continuous(
    limits = c(0, 100),
    breaks = seq(0, 100, 20),
    labels = \(x) paste0(x, "%")
  ) +
  labs(
    title = "Q4. Do you think the following services should be run in the private\nsector or
    x = "Percentage",
    y = NULL,
    fill = NULL,
    caption = "Base: All Respondents Unweighted Total: Total=1858"
  ) +
  theme_minimal() +
  theme(
    legend.position = "bottom",
    panel.grid.major.y = element_blank(),
    panel.grid.minor = element_blank()
  )
```



The improved graph incorporates a more accurate scaling and thus sends a different message by visualizing how the vast majority of people have a strong preferences for these services to be ran by the public sector.

> **Tip**
>
> Use `\n` to add a line break to your title. And note that since the title is very long, it might run off the page in your code. That's ok!
> Additionally, the colors used in the plot are `gray`, `#FF3205`, and `#006697`.

Render, commit, and push one last time. Make sure that you commit and push all changed documents and your Git pane is completely empty before proceeding.

Part 3 - DatasauRus

The data frame you will be working with in this part is called `datasaurus_dozen` and it's in the **datasauRus** package. This single data frame contains 13 datasets, designed to show us why data visualization is important and how summary statistics alone can be misleading. The different datasets are marked by the `dataset` variable, as shown in **?@fig-datasaurus**.

> **Note**
>
> If it's confusing that the data frame is called `datasaurus_dozen` when it contains 13 datasets, you're not alone! Have you heard of a baker's dozen?

Here is a peek at the top 10 rows of the dataset:

```r
datasaurus_dozen
```

```
# A tibble: 1,846 x 3
   dataset     x     y
   <chr>   <dbl> <dbl>
 1 dino     55.4  97.2
 2 dino     51.5  96.0
 3 dino     46.2  94.5
 4 dino     42.8  91.4
 5 dino     40.8  88.3
 6 dino     38.7  84.9
 7 dino     35.6  79.9
 8 dino     33.1  77.6
 9 dino     29.0  74.5
10 dino     26.2  71.4
# i 1,836 more rows
```

Question 6

In a single pipeline, calculate the mean of `x`, mean of `y`, standard deviation of `x`, standard deviation of `y`, and the correlation between `x` and `y` for each level of the `dataset` variable. Then, in 1-2 sentences, comment on how these summary statistics compare across groups (datasets).

```r
datasaurus_dozen %>%
  group_by(dataset) %>%
  summarise(
    mean_x = mean(x),
    mean_y = mean(y),
    sd_x = sd(x),
    sd_y = sd(y),
    correlation = cor(x, y)
  ) %>%
  print(n = 13)
```

```
# A tibble: 13 x 6
   dataset    mean_x mean_y  sd_x  sd_y correlation
   <chr>       <dbl>  <dbl> <dbl> <dbl>       <dbl>
 1 away         54.3   47.8  16.8  26.9     -0.0641
 2 bullseye     54.3   47.8  16.8  26.9     -0.0686
 3 circle       54.3   47.8  16.8  26.9     -0.0683
 4 dino         54.3   47.8  16.8  26.9     -0.0645
 5 dots         54.3   47.8  16.8  26.9     -0.0603
 6 h_lines      54.3   47.8  16.8  26.9     -0.0617
 7 high_lines   54.3   47.8  16.8  26.9     -0.0685
 8 slant_down   54.3   47.8  16.8  26.9     -0.0690
 9 slant_up     54.3   47.8  16.8  26.9     -0.0686
10 star         54.3   47.8  16.8  26.9     -0.0630
11 v_lines      54.3   47.8  16.8  26.9     -0.0694
12 wide_lines   54.3   47.8  16.8  26.9     -0.0666
13 x_shape      54.3   47.8  16.8  26.9     -0.0656
```

> **Tip**
>
> There are 13 groups but `tibble`s only print out 10 rows by default. Add `print(n = 13)` as the last step of your pipeline to display all rows.

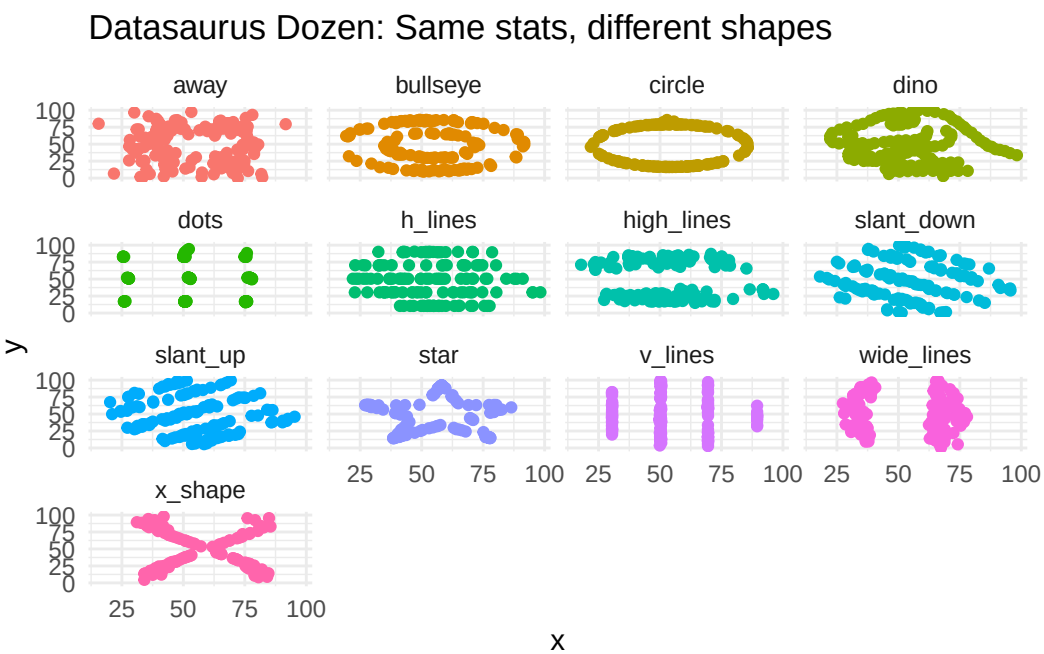
The summary statistics are nearly identical across all datasets, with the same means and standard deviations for both x and y, and very similar correlation values. Despite this, the datasets represent very different patterns when visualized, demonstrating that relying only on summary statistics can be misleading.

Question 7

Create a scatterplot of `y` versus `x` and color and facet it by `dataset`. Then, in 1-2 sentences, how these plots compare across groups (datasets). How does your response in this question compare to your response to the previous question and what does this say about using visualizations and summary statistics when getting to know a dataset?

```r
datasaurus_dozen %>%
  ggplot(aes(x = x, y = y, color = dataset)) +
  geom_point(show.legend = FALSE) +
  facet_wrap(~ dataset) +
  labs(
    title = "Datasaurus Dozen: Same stats, different shapes",
    x = "x",
```

```
  y = "y"
) +
theme_minimal()
```



The scatterplots differ dramatically across datasets, forming distinct shapes such as lines, curves, and even recognizable figures like a dinosaur and a star, despite having nearly identical summary statistics. Compared to the previous question, this shows that while summary statistics suggest the datasets are similar, visualizations reveal important structural differences, highlighting the importance of using both when analyzing data.

> **Tip**
>
> When you both color and facet by the same variable, you'll end up with a redundant legend. Turn off the legend by adding `show.legend = FALSE` to the geom creating the legend.

Render, commit, and push one last time. Make sure that you commit and push all changed documents and your Git pane is completely empty before proceeding.

Grading

The lab is graded out of a total of 28 points.

You can earn up to 4 points on each question:

- 4: Response shows excellent understanding and addresses all or almost all of the rubric items.
- 3: Response shows good understanding and addresses most of the rubric items.
- 2: Response shows understanding and addresses a majority of the rubric items.
- 1: Response shows effort and misses many of the rubric items.
- 0: Response does not show sufficient effort or understanding and/or is largely incomplete.